# CORS 安全指南

文档版本：v1.0

日期：2026-04-20

适用范围：PropOS 后端服务（Dart + Shelf）、Admin 前端（Vue 3）、uni-app H5 模式

---

## 1. 什么是 CORS

CORS（Cross-Origin Resource Sharing，跨源资源共享）是浏览器强制执行的安全机制。当一个网页向**不同源**（协议 + 域名 + 端口三者任一不同）的服务器发起请求时，浏览器会先检查服务器是否明确允许该来源，若不允许则拒绝脚本读取响应。

> 重要：CORS 是浏览器行为，服务器仍会处理并返回响应。它只阻止浏览器脚本读取结果，**不能替代服务器端的身份认证和权限控制**。

### 1.1 同源与跨源示例

| 请求来源 | 目标 URL | 是否跨源 | 原因 |
|---|---|---|---|
| `http://localhost:5173` | `http://localhost:8080/api/...` | ✅ 跨源 | 端口不同 |
| `https://admin.propos.example` | `https://api.propos.example/api/...` | ✅ 跨源 | 子域名不同 |
| `https://admin.propos.example` | `https://admin.propos.example/api/...` | ❌ 同源 | 完全相同 |
| `http://admin.propos.example` | `https://admin.propos.example/api/...` | ✅ 跨源 | 协议不同 |

---

## 2. CORS 能防什么，不能防什么

### 2.1 CORS 能防御的攻击

**跨站读取攻击（CSRF 数据窃取变种）：**

用户已登录 `admin.propos.example`，访问了恶意页面 `evil.com`。若没有 CORS 保护：

```
evil.com 脚本:
  fetch("https://api.propos.example/api/contracts")
  → 浏览器携带已有的 Cookie / Authorization Token
  → 服务器正常响应
  → evil.com 读取到租户合同数据 ❌
```

有 CORS 白名单保护后：

```
evil.com 不在白名单
  → 浏览器拒绝 evil.com 脚本读取响应内容
  → 数据窃取失败 ✅
```

### 2.2 CORS 不能防御的情况

| 攻击类型 | 原因 | 正确防御手段 |
|---|---|---|
| curl / Postman 直接调用 | 不经过浏览器，无同源策略 | JWT 认证 + RBAC |
| 移动端 App 恶意调用 | 原生 HTTP 无 CORS | JWT 认证 |
| 服务器端请求伪造（SSRF） | 服务器发出请求，不经浏览器 | 网络层隔离 |
| XSS 注入后执行脚本 | 脚本已在同源页面执行 | CSP 内容安全策略 |
| 传统表单提交 CSRF | 表单提交不受 CORS 约束 | CSRF Token |

---

## 3. PropOS 的 CORS 实现

### 3.1 中间件位置

```
请求进入
  └─ errorHandler          ← 最外层，捕获所有异常
      └─ corsMiddleware     ← CORS 必须在 auth 之前（OPTIONS 预检不带 JWT）
          └─ logMiddleware
              └─ rateLimitMiddleware
                  └─ authMiddleware    ← JWT 验证
                      └─ rbacMiddleware
                          └─ auditMiddleware
                              └─ router
```

CORS 中间件必须位于 `authMiddleware` 之前，原因：浏览器在实际请求前发送 OPTIONS 预检请求，该请求不携带 `Authorization` 头，若先经过 `authMiddleware` 会被拒绝返回 401，导致所有跨域请求失败。

### 3.2 处理逻辑

```
请求到达 corsMiddleware
  │
  ├─ 白名单为空?
  │    └─ 不添加任何 CORS 头，透传请求（纯移动端模式）
  │
  ├─ 请求 Origin 在白名单中?
  │    ├─ OPTIONS 预检 → 204 直接返回，携带 CORS 头
  │    └─ 实际请求 → 交给后续中间件，响应追加 CORS 头
  │
```

```
└─ 请求 Origin 不在白名单?
    ├─ OPTIONS 预检 → 403 直接拒绝
    └─ 实际请求 → 正常处理但不添加 CORS 头（浏览器侧拒绝读取）
```

### 3.3 响应头说明

| 响应头 | 值 | 说明 |
|---|---|---|
| `Access-Control-Allow-Origin` | 精确反射请求 Origin 或 `*` | 精确匹配时只返回该来源 |
| `Access-Control-Allow-Methods` | `GET, POST, PUT, PATCH, DELETE, OPTIONS` | 允许的 HTTP 方法 |
| `Access-Control-Allow-Headers` | `Authorization, Content-Type, X-Request-ID` | 允许的请求头 |
| `Access-Control-Max-Age` | `86400`（24 小时） | 预检结果缓存时长 |
| `Access-Control-Allow-Credentials` | `true`（仅精确匹配时） | 允许携带 Cookie / Authorization |

> 注意：`Allow-Origin: *` 时不能同时设置 `Allow-Credentials: true`，浏览器会拒绝。PropOS 实现已正确处理此约束。

---

## 4. 配置方法

通过环境变量 `CORS_ORIGINS` 配置（逗号分隔，精确匹配）：

### 4.1 本地开发（`.env` 文件）

```
# Admin Vite dev server + uni-app H5 dev server
CORS_ORIGINS=http://localhost:5173,http://localhost:3000
```

### 4.2 生产环境

```
# 只允许 Admin 前端域名
CORS_ORIGINS=https://admin.propos.example

# 同时允许 Admin + uni-app H5
CORS_ORIGINS=https://admin.propos.example,https://app.propos.example
```

### 4.3 各场景推荐值

| 部署场景 | 推荐值 | 说明 |
| --- | --- | --- |
| 生产（仅移动端 App） | 不设置（留空） | Flutter + 小程序不受 CORS 约束 |
| 生产（有 Web 前端） | 精确域名 | `https://admin.propos.example` |
| 预发/测试环境 | 精确域名 | `https://admin-staging.propos.example` |
| 本地开发 | `http://localhost:5173,http://localhost:3000` | Vite + uni-app H5 |
| 公开只读 API | `*` | 慎用，启动时会打印 `[WARN]` 提醒 |

---

## 5. 安全规则与禁止事项

### ✅ 应该做

- **精确匹配**：白名单填完整 URL（含协议和端口），不使用通配子域如 `*.propos.example`
- **HTTPS only**：生产环境白名单只填 `https://` 开头的域名
- **最小化原则**：只添加确实需要跨域访问的前端域名

### ❌ 禁止做

| 禁止行为 | 风险 |
| --- | --- |
| 生产环境设置 `CORS_ORIGINS=*` | 允许任意网站跨域读取数据 |
| 将 `http://` 域名加入生产白名单 | 流量可被中间人劫持后伪造 Origin |
| 在代码中硬编码 Origin 白名单 | 无法按环境差异配置 |
| 将后端自身端口（8080）加入白名单 | 同源请求不经过 CORS，加入无意义且混淆配置意图 |

---

## 6. CORS 与其他安全机制的关系

CORS 是浏览器侧的最后一道防线，不能独立承担安全职责，必须与以下机制配合：

```
┌──────────────────────────────────────┐
│              请求安全防护层次          │
├──────────────────────────────────────┤
│ 网络层  │ 防火墙 / VPC / IP 白名单     │
│ 传输层  │ TLS/HTTPS（防中间人）        │
```

```
│ 身份认证 │ JWT（authMiddleware）       │
│ 权限控制 │ RBAC（rbacMiddleware）      │
│ 频率限制 │ rateLimitMiddleware                 │
│ 浏览器防护│ CORS（corsMiddleware）← 本文范围 │
│ 内容安全 │ CSP 响应头（待实现）        │
└──────────────────────────────────────┘
```

**CORS 只保护浏览器用户免受恶意第三方网页的数据窃取。移动端用户的数据安全完全依赖 JWT + RBAC。**

---

## 7. 常见问题

**Q：为什么 Postman 可以访问，但前端报 CORS 错误？**

A：Postman 是原生工具，不受 CORS 约束。前端报错说明 Origin 不在白名单，检查 `CORS_ORIGINS` 配置是否包含前端实际访问的地址（注意端口）。

**Q：OPTIONS 请求返回 403，但实际请求可以发出？**

A：不可能。OPTIONS 403 说明 Origin 不在白名单，后续实际请求的 CORS 头也不会附带，浏览器会阻止脚本读取响应。

**Q：设置了 `CORS_ORIGINS` 后，移动端 App 会受影响吗？**

A：不会。Flutter 原生 HTTP 和微信小程序网络请求不经过浏览器，CORS 机制对它们完全透明。

**Q：为什么 `Allow-Origin: 不能和 Allow-Credentials: true` 同时用？**

A：这是浏览器规范的强制要求（RFC 6454）。允许通配符来源时附带凭证意味着任何网站都能以用户身份发请求，浏览器直接拒绝此组合以防止滥用。

---

文档维护：后端团队 / 对应代码：`lib/core/middleware/cors_middleware.dart`